

EduGuard - Demo Technical Guide

Backend + Frontend reference for answering technical questions during the demo

0. The 30-Second Pitch

EduGuard (EduSiksha) is a safe, parent-controlled AI tutor for kids.

- Kids chat with an AI tutor locked to school subjects only (guardrails).
- Kids can upload study material (PDF, Word, images, etc.) and ask questions about it - the AI answers using that material (RAG).
- Parents get a dashboard to: control access (on/off switch), read every chat transcript (safety audit), see AI-generated learning analytics (mastery score, topics), download PDF report cards / practice papers, and pay via Razorpay.

Tech in one line: Next.js frontend communicates with a FastAPI backend (REST + Socket.IO WebSockets), backed by PostgreSQL + pgvector, using local embeddings and Groq's Llama 3.3 for generation.

1. System Architecture (High Level)

```
Next.js Frontend (React 19, port 3000)
| REST (HTTP, axios) --> FastAPI Backend (port 8000)
| Socket.IO (WebSocket) <--> REST routers + Socket.IO server
```

FastAPI talks to:

- PostgreSQL + pgvector (768-d) : RAG + records
- SentenceTransformer all-mpnet-base-v2 : embeddings, LOCAL on CPU
- Groq Cloud llama-3.3-70b-versatile : text generation
- Razorpay : payments

Key idea to say out loud: "Embeddings run locally and free on the CPU; only the final text generation is sent to Groq. We never send the child's documents to an external embedding API."

2. Tech Stack (exact names/versions)

Layer	Technology	Notes
Frontend	Next.js 16 (App Router), React 19, TypeScript	frontend/
Styling	Tailwind CSS v4	utility classes inline
Realtime	socket.io-client 4.8	chat is over WebSockets
HTTP client	axios	lib/api.ts, auto-injects JWT
Markdown/Math	react-markdown, remark-math, rehype-katex, rehype-highlight	renders AI answers, LaTeX, code
i18n	i18next / react-i18next	English, Hindi, Kannada
Backend	FastAPI (Python), Uvicorn ASGI	backend/app/
Realtime server	python-socketio (AsyncServer, ASGI)	services/websocket.py
ORM	SQLAlchemy	db/models.py
Database	PostgreSQL + pgvector	vector similarity search
Embeddings	SentenceTransformers all-mpnet-base-v2 -> 768 dims, on CPU	services/ai.py (LOCAL, free)
LLM	Groq API, model llama-3.3-70b-versatile	generation + analytics + paper gen
Auth	JWT (python-jose, HS256) + Argon2 hashing (passlib)	core/security.py
Docs/OCR	pdfplumber, pytesseract, pdf2image, pandas, python-pptx, python-docx, Pillow	services/document_processor.py
PDF output	ReportLab	services/pdf_generator.py
Payments	Razorpay	services/payment.py

3. Backend Walkthrough (backend/app/)

3.1 Entry point - main.py

- Creates the FastAPI app, enables CORS for `http://localhost:3000` and `:8000`.
- Mounts routers: `/auth`, `/ (data)`, `/chat`, `/payment`.
- Serves generated PDFs as static files at `/static/reports`.
- Wraps FastAPI with Socket.IO ASGI (`socket_app`) at path `/socket.io` - so HTTP and WebSocket share one server on port 8000.
- Background cleanup task on startup: runs `cleanup_expired_files()` once every 24h.
- Run command: `uvicorn app.main:socket_app --host 0.0.0.0 --port 8000 --reload`.

3.2 Database - db/base.py + db/models.py

`base.py`: SQLAlchemy engine from `DATABASE_URL`, `SessionLocal`, and `get_db()` dependency (one session per request).

Table	Purpose / key columns
User	Parent: email (unique), phone, password_hash. Has many Kid.
Kid	username (unique), password_hash, subscription_status, subscription_expiry, is_active_access (parental on/off), parent_id.
ChatSession	kid_id, title (first 30 chars of first message), is_active, timestamps.
Message	session_id, role ('user' or 'ai'), content, timestamp.
Document	RAG chunk store: session_id, file_path, file_name, content (chunk text), embedding = Vector(768), is_active, created_at.
ChatAnalytics	One row per session: mastery_score (0-100), topics (JSONB array), summary, language, last_updated. Cache for analytics.

3.3 Authentication - api/auth.py + core/security.py

- Passwords hashed with Argon2 (passlib); never stored in plain text.
- JWT signed with `SECRET_KEY`, HS256, default expiry 30 minutes.
- Token payload carries id, type ('parent' or 'kid'), sub (email/username).
- Guards: `get_current_user_id` (requires parent) and `get_current_kid_id` (requires kid).
- Endpoints: `POST /auth/parent/signup`, `/auth/parent/login`, `/auth/kid/create` (parent-only), `/auth/kid/login`.
- Role separation enforced at the token level - a kid token cannot hit parent endpoints and vice versa.

3.4 Document ingestion / RAG indexing - document_processor.py + api/chat.py

Upload flow (POST `/chat/upload`):

1. File saved to `backend/uploads/<timestamp>_<filename>`.
2. `process_document()` routes by extension: Images -> Tesseract OCR; PDF -> pdfplumber, with OCR fallback (<100 chars) via pdf2image at 300 DPI; Excel -> pandas; PowerPoint -> python-pptx; Word -> python-docx; TXT -> raw read.
3. `smart_chunk()` splits text into ~500-char chunks with 50-char overlap, respecting paragraph/sentence boundaries.
4. Each chunk is embedded locally (all-mpnet-base-v2, 768-d) and stored as a Document row with its vector.

3.5 RAG retrieval + LLM generation - services/ai.py

`generate_response(kid_id, message, chat_history, db)`:

- 1. Embed the user's question locally -> query vector.
- 2. pgvector cosine search (Document.embedding.cosine_distance), take top 2 chunks -> context.
- 3. Build payload: system guardrail prompt -> optional RAG context -> chat history -> user message.
- 4. Call Groq llama-3.3-70b-versatile, temperature=0.5, max_tokens=500. Return the answer.

Guardrail prompt (the safety core): instructs the model to ONLY answer school subjects (Math, Science, History, Coding, language). Off-topic requests get: "This topic is outside our educational context. Let's get back to your studies!"

Privacy cleanup - cleanup_expired_files(): documents older than 15 days have their file deleted from disk, embedding nulled, and is_active set to False. Runs daily.

3.6 Realtime chat - services/websocket.py

Socket.IO AsyncServer, CORS *. On chat_message event with {kid_id, message, session_id}:

- 1. Find the session by session_id (scoped to kid_id); if none, create a new session titled from the first message.
- 2. Save the user message.
- 3. Load up to the last 100 messages of that session as context.
- 4. Call generate_response() (RAG + Groq).
- 5. Save the AI message.
- 6. Emit response back to that socket with {data, session_id, title}.

3.7 Parent data + analytics - api/data.py

- GET /parent/dashboard/kids - parent's kids + access status.
- GET /kid/chats - a kid's own sessions.
- GET /parent/kid/{kid_id}/chats - parent views a kid's sessions (ownership checked).
- GET /kid/chats/{chat_id}/messages - transcript (ownership checked).
- GET /parent/chat/{chat_id}/stats - analytics engine: returns cached ChatAnalytics if present for the language; otherwise sends the student's questions to Groq (JSON mode) for mastery_score (0-100), exactly 5 topics, and a summary in the requested language. Caches result.
- POST /parent/chat/{chat_id}/stats/refresh - clears cache and regenerates.
- POST /reports/generate/{chat_id} - generates practice paper + answer key PDFs, returns /static/reports/{id}_qp.pdf and _key.pdf.

3.8 PDF generation - services/pdf_generator.py

- Sends chat history to Groq -> JSON array of 10-15 practice questions with answers (recall -> comprehension -> application).
- ReportLab builds two PDFs: a Question Paper (no answers) and an Answer Key. Fallback questions if the AI call fails.

3.9 Payments - api/payment.py + services/payment.py

- POST /payment/create-order - creates a Razorpay order, default Rs.499 (49900 paise), returns order_id, amount, key_id.
- POST /payment/verify - verifies the Razorpay HMAC signature (order_id, payment_id, signature).

4. Frontend Walkthrough (frontend/)

4.1 Pages (app/)

- page.tsx - public landing page (features, pricing).

- login/, signup/ - parent auth forms. JWT saved to localStorage (token, user_type).
- chat/page.tsx - the kid chat app (Gemini-style UI): connects to Socket.IO; decodes JWT client-side for kid_id; fetches /kid/chats; emits chat_message and listens for response; uploads files (10 MB max) to /chat/upload; renders AI replies as Markdown with KaTeX math and syntax-highlighted code.
- dashboard/page.tsx - parent home: kid cards, access toggle, subscription.
- dashboard/kid/[id]/page.tsx - a kid's activity (sessions, message counts).
- dashboard/chat/[id]/page.tsx - session inspector: mastery charts, AI analytics, full transcript (safety audit), generate report card button.

4.2 Shared (components/, lib/)

- SocketProvider.tsx - React Context creating one socket.io-client connection to http://localhost:8000.
- lib/api.ts - axios instance with an interceptor that auto-adds Authorization: Bearer <token> from localStorage.
- lib/i18n.ts - translations for English, Hindi, Kannada.
- middleware.ts - currently a placeholder pass-through (auth enforced client-side because the token lives in localStorage).

5. End-to-End Flow: Kid uploads notes, then asks a question

1. Kid uploads notes.pdf -> POST /chat/upload (axios, multipart)
2. Backend extracts text -> pdfplumber / OCR fallback
3. Chunk (~500 chars) -> smart_chunk()
4. Embed each chunk LOCALLY -> all-mpnet-base-v2 (768-d)
5. Store chunks + vectors -> Document table (pgvector)
6. Kid asks 'What is X?' -> Socket.IO emit 'chat_message'
7. Embed question locally -> query vector
8. pgvector cosine search -> top-2 matching chunks
9. Build prompt: guardrail + context + history(<=100) + question
10. Groq llama-3.3-70b -> answer (temp 0.5, max 500 tokens)
11. Save user + AI messages -> Message table
12. Socket.IO emit 'response' -> frontend renders Markdown

6. Likely Demo Questions (with crisp answers)

Q: Which AI model do you use?

A: Groq's llama-3.3-70b-versatile for generation, analytics, and practice-paper creation. Embeddings use a local SentenceTransformer model (all-mpnet-base-v2).

Q: Is this just ChatGPT with a wrapper?

A: No. It's RAG over the child's own study material + a strict educational guardrail + a parent oversight layer (transcripts, analytics, access control). The chat is grounded in uploaded documents, not generic web knowledge.

Q: How do you keep it safe for kids?

A: Four layers: (1) guardrail system prompt restricts answers to school subjects; (2) parents read every transcript; (3) parental access toggle (is_active_access) can cut off the chatbot; (4) 15-day auto-deletion of uploaded files for privacy.

Q: What's RAG and how is it implemented here?

A: Retrieval-Augmented Generation. We embed uploaded documents into vectors, store them in PostgreSQL + pgvector, and at query time do a cosine-similarity search for the top-2 relevant chunks, injected into the LLM prompt as context.

Q: Why local embeddings instead of an API?

A: Privacy + cost. Children's documents never leave our server for embedding, and it's free (runs on CPU). Only the final question + retrieved context goes to Groq for text generation.

Q: Why 768 dimensions?

A: all-mpnet-base-v2 outputs 768-dim vectors, and the Document.embedding column is Vector(768) to match. Changing the model means changing dimensions -> DB migration.

Q: How does real-time chat work?

A: Socket.IO over WebSockets. The frontend emits chat_message; the backend processes (save -> RAG -> Groq -> save) and emits response. REST and WebSocket share one server on port 8000.

Q: How is authentication handled?

A: JWT (HS256, 30-min expiry) with Argon2-hashed passwords. The token encodes a type (parent/kid) so role-based route guards keep kids out of parent endpoints.

Q: What file types can kids upload?

A: Images (PNG/JPG/BMP/TIFF via OCR), PDF (with OCR fallback for scans), Word, Excel, PowerPoint, and TXT - max 10 MB.

Q: How does the analytics / mastery score work?

A: We send the student's questions to the LLM and ask it to score question quality/depth (0-100), extract exactly 5 topics, and write a summary in the chosen language. Results are cached per session in ChatAnalytics.

Q: Multilingual?

A: UI supports English, Hindi, Kannada (i18next), and analytics summaries are generated in the selected language.

Q: How do payments work?

A: Razorpay - backend creates an order (Rs.499 default), frontend completes payment, backend verifies the HMAC signature.

7. Honest Gotchas (so a sharp reviewer can't surprise you)

These are real characteristics of the current code. If asked, frame them as 'known, on the roadmap.'

- RAG search is not session-scoped: generate_response queries the top-2 Document chunks across ALL documents, not filtered by the current kid/session. Roadmap: filter by session_id + is_active.
- WebSocket connection is unauthenticated: the Socket.IO connect handler has a TODO - kid_id is trusted from the client payload. Roadmap: validate JWT on connect.
- History context comment mismatch: code loads the last 100 messages (a comment says 20). Say 'up to 100.'
- Payment verify doesn't persist the subscription yet: /payment/verify confirms the signature but doesn't flip subscription_status / is_active_access in the DB. Roadmap: update the Kid record on success.
- Token in localStorage + client-side auth: middleware.ts is a pass-through; route protection happens in each page. Trade-off chosen for simplicity.
- Hardcoded localhost:8000 in api.ts and SocketProvider.tsx. Roadmap: environment variables for production.
- Tesseract path is hardcoded to C:\Program Files\Tesseract-OCR\tesseract.exe (Windows). Needs to be configurable for deployment.
- CORS allow_origins=* on Socket.IO - fine for dev, tighten for prod.

8. One-Liners to Memorize

- 'Local embeddings, cloud generation' - privacy-first split.
- 'pgvector cosine search, top-2 chunks' - the RAG core.
- 'Guardrail prompt + parent transcripts + access toggle + 15-day deletion' - the safety story.
- 'One FastAPI server hosts both REST and Socket.IO on port 8000.'
- 'Argon2 passwords, JWT with role claims.'
- 'Llama 3.3 70B on Groq for everything generative.'